Task 1: GUI Design Review – Fee Payment Window

Task 1.1 – Diagnose

The design mixes GUI, validation and database logic; compares Strings using ==; accepts empty or invalid amounts; performs database work and sleep on the event-dispatch thread; builds SQL by concatenation (SQL injection risk); does not implement Receipt correctly; gives weak user feedback; does not reliably close database resources or handle checked exceptions; and has no proper window-closing handler.

Task 1.2 – Repair

Use FeeWindow for input and messages, FeeService for validation/payment rules, and FeeRepository for JDBC. Use PreparedStatement and try-with-resources in the repository. Run slow database work in SwingWorker so the window remains responsive. The source below demonstrates the separated GUI/service design; connect FeeService to a repository for a real database.

Task 1.3 – Defence

The redesigned classes have separate responsibilities. FeeService can be tested without opening a window, and the window can be changed without rewriting payment logic. Database code is isolated, easier to secure and easier to maintain.

Alternative and complexity

A chain checking k action commands may perform up to O(k) comparisons. A separate listener or Swing Action routes an event directly, approximately O(1) application-level dispatch. k distinct listener objects use O(k) additional space. A shared Swing Action can serve buttons, menus and keyboard shortcuts.

Java Source Code

```
import java.awt.*;
import java.awt.event.*;

class FeeService {
    String pay(String id, String text) {
        if (id.trim().isEmpty()) throw new IllegalArgumentException("Enter registration number");
        double amount;
        try { amount = Double.parseDouble(text); }
        catch (NumberFormatException e) { throw new IllegalArgumentException("Enter valid amount"); }
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        return "Payment successful: Rs. " + amount;
    }
}
public class FeeWindow extends Frame {
    TextField id = new TextField(), amount = new TextField();
    Label status = new Label("Enter fee details");
    FeeService service = new FeeService();
    public FeeWindow() {
        super("Student Fee Payment");
        setLayout(new GridLayout(4,2,5,5));
        add(new Label("Registration No")); add(id);
        add(new Label("Amount")); add(amount);
```

```
        Button pay = new Button("Pay"), clear = new Button("Clear"), receipt = new Button("Receipt");
        add(pay); add(clear); add(receipt); add(status);
        pay.addActionListener(e -> { try { status.setText(service.pay(id.getText(), amount.getText())); }
            catch (IllegalArgumentException ex) { status.setText(ex.getMessage()); }});
        clear.addActionListener(e -> { id.setText(""); amount.setText(""); status.setText("Cleared"); });
        receipt.addActionListener(e -> status.setText("Receipt for " + id.getText()));
        addWindowListener(new WindowAdapter(){ public void windowClosing(WindowEvent e){dispose();}});
        setSize(420,190);
    }
    public static void main(String[] args){new FeeWindow().setVisible(true);}
}
```

Input Files

No external input file is required; enter values in the GUI.

Output Screenshot

Illustrative console-style output in the requested black background, white text and green status theme. For GUI tasks, add genuine screenshots of the actual window and interactions.



Compile and Run

Open terminal in src, then run:
javac FeeWindow.java
java FeeWindow